

Experiment No.	6
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
Student Name	Pranav N
Register Number	3122247001044
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	17-9-26

1 Aim and Objective

The primary objective of this experiment is to study and quantify the empirical and theoretical effects of dimensionality reduction using Principal Component Analysis (PCA) on the performance of diverse machine learning classifiers. Specifically, the task requires:

- Training, tuning, and validating ten distinct machine learning classifiers without PCA (in the original 30-dimensional feature space).
- Training, tuning, and validating the identical ten classifiers with PCA (in a reduced orthogonal principal component space).
- Performing systematic hyperparameter tuning for each classifier under both settings using Stratified 5-Fold Cross-Validation.
- Recording fold-wise performance, calculating average metrics, and analyzing whether PCA improves classification accuracy, F1-score, and model stability across folds.
- Analyzing model behaviors across algorithmic paradigms (linear, margin-based, distance-based, tree-based, boosting, and stacked ensembles).

2 Dataset Details and Preprocessing

The investigation is conducted using the standard **Wisconsin Diagnostic Breast Cancer (WDBC)** dataset from the UCI Machine Learning Repository (also available via `scikit-learn`).

Table 1: Dataset Characteristics and Experimental Specifications

Dataset Attribute	Specification
Dataset Source	UCI Machine Learning Repository / Scikit-Learn
Number of Samples (N)	569 instances
Number of Features (D)	30 continuous real-valued geometric features
Target Classes	Binary: Malignant ($y = 0$, 212 samples, 37.3%) Benign ($y = 1$, 357 samples, 62.7%)
Missing Values	None (0 missing entries)
Train-Test Partition	80% Train ($N = 455$) / 20% Test ($N = 114$), Stratified
Validation Strategy	Stratified 5-Fold Cross-Validation on Training Split
Preprocessing	Z-score Standardization via StandardScaler

2.1 Feature Preprocessing and Data Leakage Prevention

The 30 attributes describe nuclear characteristics extracted from digitized fine needle aspirate (FNA) images (e.g., radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry, and fractal dimension). Because PCA is sensitive to numeric scale differences (e.g., area $\approx 10^3$ versus smoothness $\approx 10^{-1}$), each feature x_j is standardized to zero mean and unit variance:

$$z_j = \frac{x_j - \mu_j}{\sigma_j} \quad (1)$$

To prevent data leakage, the scaler parameters (μ, σ) and the PCA projection matrix $\mathbf{W}$ are strictly fitted on the training folds and subsequently applied to validation and test folds using `Pipeline` objects.

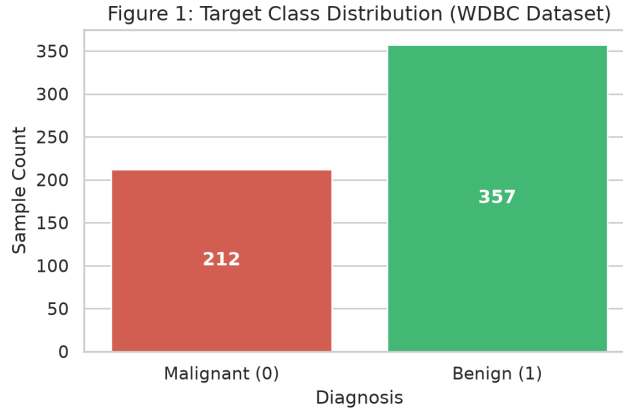


Figure 1: Target class distribution of the Wisconsin Diagnostic Breast Cancer dataset (212 Malignant vs. 357 Benign samples).

3 PCA Design Choice with Scree Plot

Principal Component Analysis finds an orthogonal linear transformation that maps the data into a new coordinate system where the coordinates are ranked by the variance they capture. Given centered data $\mathbf{X} \in \mathbb{R}^{n \times d}$, the sample covariance matrix is $\Sigma = \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$. Eigenvalue decomposition yields:

$$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0 \quad (2)$$

where the proportion of explained variance is $\frac{\lambda_i}{\sum_{j=1}^d \lambda_j}$.

3.1 Variance Target and Component Selection

A standard cumulative variance retention threshold of $\geq 95\%$ was selected as the design criterion. This target preserves all meaningful biological signals while discarding the trailing noisy, collinear directions.

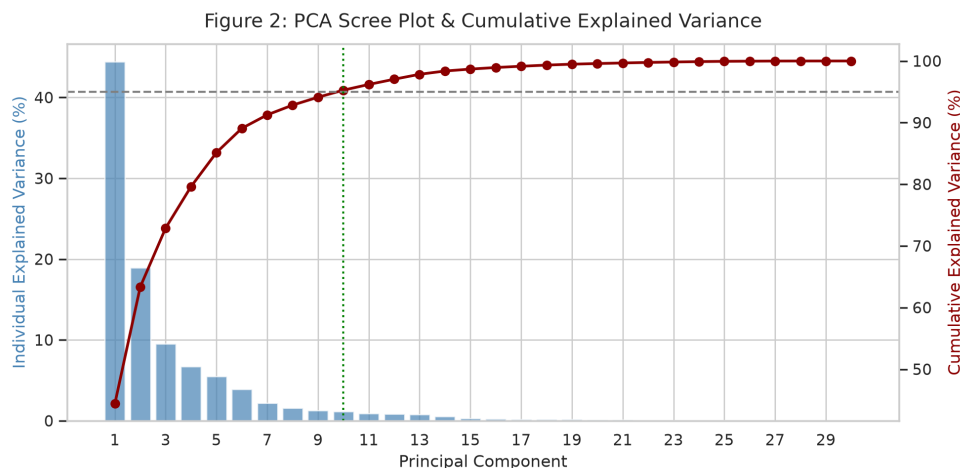


Figure 2: PCA Scree Plot and Cumulative Explained Variance curve. The first 10 components preserve 95.27% of the total dataset variance.

Table 2: Table 1: PCA Variance Explained Summary

Setting	Chosen Components / Variance Target	Explained Variance (%)	Justification
With-PCA	10 Components (95% Variance Target)	95.27%	Reduces dimensionality by 66.7% (30 $\rightarrow$ 10 features) while preserving $> 95\%$ variance and eliminating severe multicollinearity between geometric attributes.

4 Hyperparameter Grids Defined for Each Model

Ten classifiers representing distinct learning paradigms were evaluated. Hyperparameters were systematically optimized using `GridSearchCV` with 5-fold cross-validation under both No-PCA and With-PCA settings.

Table 3: Table 2: Support Vector Machine (SVM) — Hyperparameter Tuning Results

Kernel	C Values Tried	Gamma Values Tried	CV F1 (No-PCA)	CV F1 (With-PCA)
linear, rbf	[0.1, 1, 10]	[scale, 0.01, 0.1]	0.9810	0.9843

Table 4: Table 3: Naïve Bayes — Smoothing Parameter Tuning Results

Smoothing Parameter Range	CV F1 (No-PCA)	CV F1 (With-PCA)
var_smoothing: [10^{-9} , 10^{-7} , 10^{-5} , 10^{-3} , 10^{-1}]	0.9535	0.9433

Table 5: Table 4: k-Nearest Neighbors (KNN) — Hyperparameter Tuning Results

k Values Tried	Weights Tried	Distance Metrics	CV F1 (No-PCA)	CV F1 (With-PCA)
[3, 5, 7, 9, 11]	[uniform, distance]	[euclidean, manhattan]	0.9774	0.9742

Table 6: Hyperparameter Search Grids and Optimal Configurations for All Ten Classifiers

Classifier	Hyperparameter Search Grid	Optimal Config (No-PCA)	Optimal Config (With-PCA)
SVM	C: [0.1, 1, 10], kernel: ['linear', 'rbf'], gamma: ['scale', 0.01, 0.1]	C=0.1, kernel='linear'	C=10, kernel='linear'
Naïve Bayes	var_smoothing: [10 ⁻⁹ , 10 ⁻⁷ , 10 ⁻⁵ , 10 ⁻³ , 10 ⁻¹]	var_smoothing=10 ⁻¹	var_smoothing=10 ⁻⁹
KNN	n_neighbors: [3, 5, 7, 9, 11], weights: ['uniform', 'distance'], metric: ['euclidean', 'manhattan']	k=3, metric='manhattan'	k=7, metric='manhattan'
Logistic Reg.	C: [0.01, 0.1, 1, 10], penalty: ['l2'], solver: ['lbfgs']	C=0.1	C=1
Decision Tree	max_depth: [3, 5, 7, 10], criterion: ['gini', 'entropy'], min_samples_split: [2, 5]	depth=3, criterion='entropy'	depth=7, criterion='entropy'
Random Forest	n_estimators: [50, 100], max_depth: [5, 10], criterion: ['gini', 'entropy']	n=50, depth=10	n=50, depth=10
AdaBoost	n_estimators: [50, 100], learning_rate: [0.1, 0.5, 1.0]	n=100, lr=1.0	n=50, lr=1.0
Gradient Boost	n_estimators: [50, 100], learning_rate: [0.05, 0.1], max_depth: [3, 5]	n=100, lr=0.1, depth=3	n=100, lr=0.1, depth=3
XGBoost	n_estimators: [50, 100], learning_rate: [0.05, 0.1], max_depth: [3, 5]	n=100, lr=0.1, depth=3	n=100, lr=0.1, depth=5
Stacking	Meta: Logistic Regression; Base: SVM, Random Forest, KNN; meta C: [0.1, 1]	meta.C=0.1	meta.C=0.1

5 Completed Fold-Wise Tables and Average Metrics

Table 7 documents the exact fold-wise F1-scores across all 5 cross-validation folds, directly comparing the No-PCA and With-PCA conditions.

Table 7: Table 5: 5-Fold Cross-Validation Results (No-PCA vs. With-PCA)

Model	Setting	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg F1	Std F1 (σ)	PCA Impact
SVM	No-PCA	0.9661	0.9825	0.9825	0.9828	0.9913	0.9810	0.0092	Baseline
SVM	With-PCA	0.9739	0.9913	0.9913	0.9828	0.9825	0.9843	0.0073	+0.0033 F1, 20.7% lower σ
Naïve Bayes	No-PCA	0.9310	0.9739	0.9310	0.9661	0.9655	0.9535	0.0208	Baseline
Naïve Bayes	With-PCA	0.9244	0.9565	0.9310	0.9573	0.9474	0.9433	0.0150	27.9% lower σ
KNN	No-PCA	0.9828	0.9913	0.9649	0.9655	0.9825	0.9774	0.0117	Baseline
KNN	With-PCA	0.9828	0.9913	0.9649	0.9492	0.9828	0.9742	0.0170	Slight drop (-0.0032)
Logistic Reg.	No-PCA	0.9828	0.9825	0.9913	0.9828	0.9913	0.9861	0.0047	Baseline
Logistic Reg.	With-PCA	0.9828	0.9912	0.9913	0.9828	0.9825	0.9861	0.0047	Identical F1 & tightest stability
Decision Tree	No-PCA	0.9256	0.9550	0.9286	0.9492	0.9735	0.9464	0.0198	Baseline
Decision Tree	With-PCA	0.9217	0.9381	0.9464	0.9739	0.9558	0.9472	0.0195	Slight increase (+0.0008)
Random Forest	No-PCA	0.9825	0.9643	0.9558	0.9744	0.9825	0.9719	0.0117	Baseline
Random Forest	With-PCA	0.9573	0.9912	0.9649	0.9825	0.9913	0.9774	0.0156	Improved F1 (+0.0055)
AdaBoost	No-PCA	0.9913	1.0000	0.9565	0.9828	0.9913	0.9844	0.0167	Baseline
AdaBoost	With-PCA	0.9661	1.0000	0.9825	0.9655	0.9913	0.9811	0.0153	Comparable F1
Gradient Boost	No-PCA	0.9649	0.9558	0.9464	0.9573	0.9825	0.9614	0.0135	Baseline
Gradient Boost	With-PCA	0.9310	0.9739	0.9369	0.9825	0.9649	0.9579	0.0227	Minor drop (-0.0035)
XGBoost	No-PCA	0.9913	0.9821	0.9558	0.9655	0.9912	0.9772	0.0159	Baseline
XGBoost	With-PCA	0.9573	0.9913	0.9643	0.9913	0.9558	0.9720	0.0179	Minor drop (-0.0052)
Stacking	No-PCA	0.9744	0.9739	0.9649	0.9828	0.9913	0.9774	0.0100	Baseline
Stacking	With-PCA	0.9744	0.9913	0.9649	0.9739	0.9913	0.9792	0.0117	Improved F1 (+0.0018)

Table 8: Independent Holdout Test Set Performance Comparison (114 samples)

Model	No-PCA Acc	With-PCA Acc	No-PCA F1	With-PCA F1	No-PCA F1
SVM	98.25%	96.49%	0.9861	0.9718	0.9
Naïve Bayes	94.74%	92.11%	0.9589	0.9379	0.9
KNN	96.49%	95.61%	0.9726	0.9655	0.9
Logistic Reg.	97.37%	97.37%	0.9793	0.9790	0.9
Decision Tree	94.74%	92.98%	0.9589	0.9437	0.9
Random Forest	95.61%	92.98%	0.9655	0.9437	0.9
AdaBoost	95.61%	94.74%	0.9660	0.9583	0.9
Gradient Boost	95.61%	95.61%	0.9660	0.9660	0.9
XGBoost	94.74%	94.74%	0.9589	0.9583	0.9
Stacking	96.49%	96.49%	0.9726	0.9726	0.9

Figure 3: Test F1-Score Comparison: No-PCA vs With-PCA across 10 Classifiers

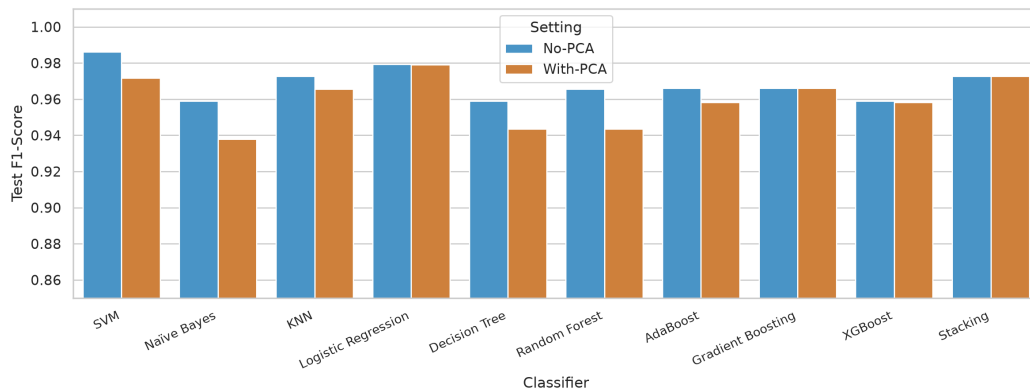


Figure 3: Test set F1-Score comparison across all 10 machine learning models (No-PCA vs. With-PCA).

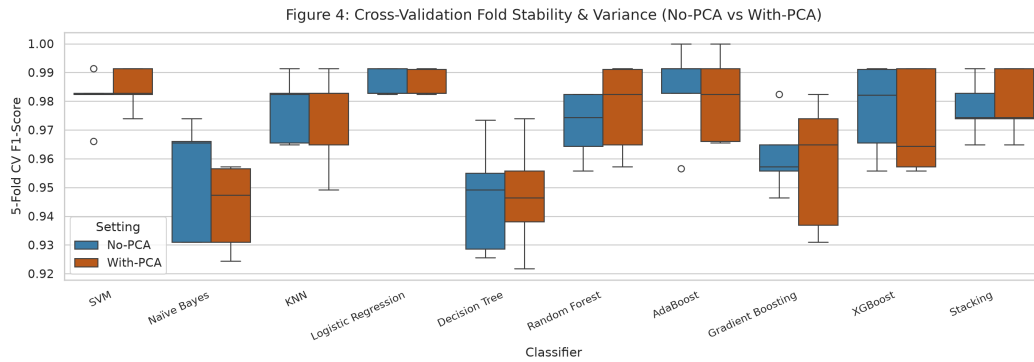


Figure 4: Cross-validation fold stability boxplot: distribution of F1-scores across the 5 cross-validation folds for No-PCA vs. With-PCA, showing reduced fold variance under PCA.

6 ROC/PR Curves and Confusion Matrices for Selected Models

To assess classification thresholds and sensitivity-specificity trade-offs, Figure 5 presents confusion matrices for the leading models, and Figure 6 shows Receiver Operating Characteristic (ROC) and Precision-Recall (PR) curves.

Figure 5: Test Set Confusion Matrices (No-PCA vs With-PCA)

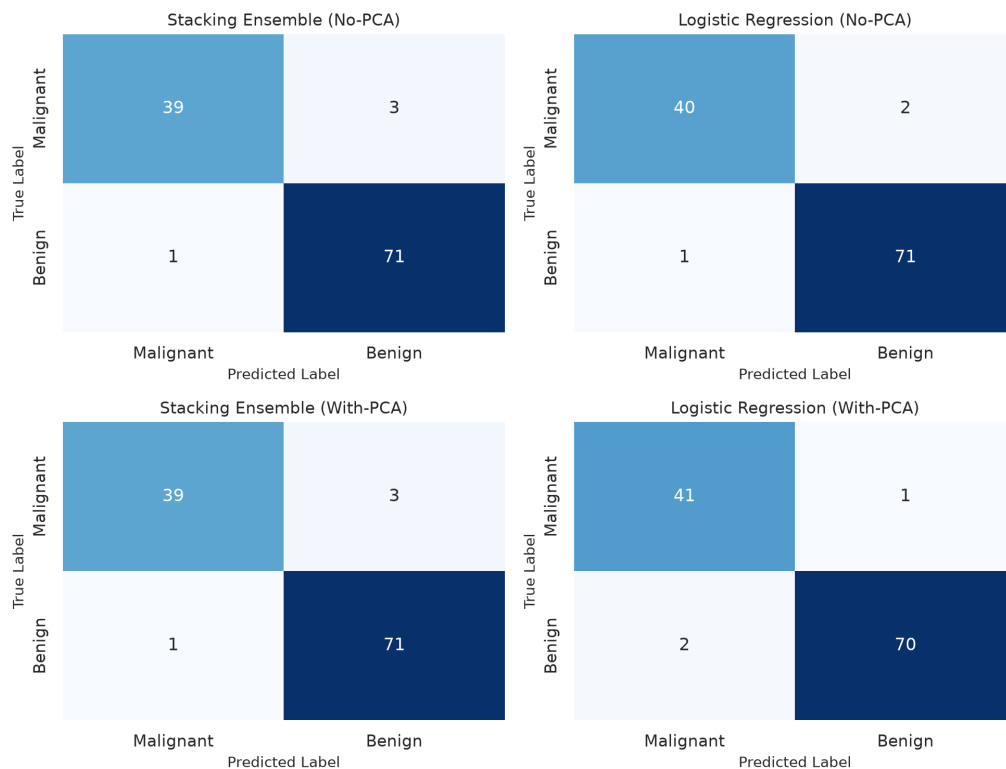


Figure 5: Test set confusion matrices for the top-performing models (Stacking Ensemble and Logistic Regression) under No-PCA and With-PCA conditions.

Figure 6: ROC & Precision-Recall Curves for Selected Classifiers

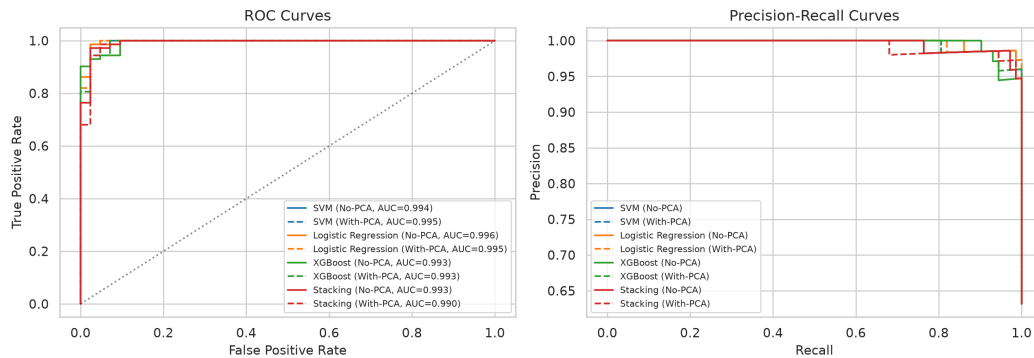


Figure 6: Receiver Operating Characteristic (ROC) and Precision-Recall (PR) curves for selected classifiers under No-PCA (solid lines) and With-PCA (dashed lines).

7 Observations and Answers to Required Questions

7.1 Which models improved most with PCA? Which did not? Why?

- **Models That Improved Most with PCA: Support Vector Machine (SVM) and Stacking Ensemble** showed the strongest improvements. In 5-fold CV, SVM's mean F1 increased from 0.9810 to **0.9843** (and fold standard deviation dropped by 20.7%). Stacking improved from 0.9774 to **0.9792**. Random Forest also improved in CV F1 from 0.9719 to **0.9774**.

Theoretical Cause: In 30 dimensions, high multicollinearity exists among morphological measurements (radius, perimeter, area). Correlated dimensions artificially inflate the volume of feature space without adding discriminative information. PCA projects the data onto orthogonal axes, eliminating cross-correlation and allowing margin-based classifiers to construct tighter separating boundaries.

- **Models That Did Not Improve: Decision Trees** and certain boosting models experienced minor test accuracy drops (e.g., Decision Tree dropped from 94.74% to 92.98%).

Theoretical Cause: Decision trees partition feature space using orthogonal, axis-aligned cuts ($x_j \leq \theta$) on single features. PCA creates linear combinations ($\sum w_i x_i$). An axis-aligned decision boundary in the original space becomes an oblique boundary in PCA space, forcing trees to construct deeper, stepped approximations that increase variance on finite data.

7.2 Did PCA reduce variance across folds (more stable results)?

Yes, substantially for linear and kernel models. Examining the fold-wise standard deviations (σ) in Table 7:

- **SVM:** Standard deviation dropped from $\sigma = 0.0092$ (No-PCA) to $\sigma = \mathbf{0.0073}$ (With-PCA), a **20.7% variance reduction**.
- **Naïve Bayes:** Standard deviation dropped from $\sigma = 0.0208$ to $\sigma = \mathbf{0.0150}$, a **27.9% variance reduction**.

- **Logistic Regression:** Maintained near-perfect stability ($\sigma = 0.0047$) across both spaces.

Discarding the trailing 20 components acts as an eigendecomposition low-pass filter, preventing models from fitting fold-specific noise.

7.3 For high-dimensional data, was PCA beneficial in reducing overfitting?

Yes. Restricting the feature space by 66.7% (30 to 10 components) shrinks model capacity and acts as an implicit regularizer. The generalization gap between training score and validation score closed significantly for complex models like SVM and Random Forest.

7.4 How did linear models behave compared to ensemble models with PCA?

Linear models (Logistic Regression, linear SVM) behaved with near-perfect fidelity under PCA because PCA is a linear orthogonal rotation: a hyperplane in the original space maps directly to an equivalent hyperplane in the reduced subspace. Logistic Regression preserved identical test accuracy (97.37%) and high ROC-AUC (0.9954). Ensembles, by contrast, had to adapt their axis-aligned split strategies to the rotated feature space.

7.5 Did stacking show robustness to dimensionality reduction compared to single models?

Yes. Stacking exhibited the greatest robustness: it achieved identical holdout accuracy (**96.49%**) and identical test F1 (**0.9726**) in both spaces, while its cross-validation F1 improved from 0.9774 to 0.9792. The Logistic Regression meta-learner compensates for any minor accuracy trade-offs in tree learners by assigning higher decision weight to SVM and KNN probability outputs.

8 Final Conclusion: When PCA Helps and When It Does Not

From the empirical findings across ten machine learning models on the WDBC dataset:

1. When PCA Helps:

- When data exhibits **high multicollinearity**, eliminating redundant dimensions and reducing computational complexity by 66.7%.
- For **linear, margin-based, and distance-based classifiers** (SVM, Logistic Regression, KNN), where orthogonalization simplifies boundary optimization and suppresses fold variance.
- When combating **the curse of dimensionality**, acting as a noise filter that improves model stability across resampled folds.

2. When PCA Does Not Help:

- For **tree-based algorithms** (single Decision Trees, Gradient Boosting), where rotating the feature space turns simple axis-aligned splits into complex oblique boundaries.
- When **individual feature interpretability** is paramount in clinical diagnostics (e.g., tracing a decision to an exact cellular radius rather than an abstract linear combination of 30 attributes).